

Biblioteca para Manipulação de Grafos Simples e Direcionados:

Relatório da Etapa 1

Luan Inaibes¹, Otávio Santos¹, Arthur Luis¹

¹Curso de Computação – Instituição de Ensino
Cidade – Estado – Brasil

kuskynz@gmail.com

Abstract.

Resumo.

1. Arquitetura e API

A biblioteca é estruturada em torno da classe abstrata `AbstractGraph`, que define a API comum, mantém os atributos compartilhados (como os pesos dos vértices), fornece métodos auxiliares (por exemplo, a validação de índices de vértices) e declara os métodos abstratos implementados pelas classes concretas. Algoritmos independentes de representação, como a verificação de conectividade e a exportação para o Gephi, são implementados uma única vez na classe abstrata.

Duas classes concretas estendem `AbstractGraph`:

- `AdjacencyMatrixGraph`: representa as arestas em uma matriz $n \times n$, em que cada posição armazena o peso da aresta correspondente, favorecendo consultas de existência e de peso em tempo constante;
- `AdjacencyListGraph`: representa as arestas por listas de adjacência (dicionários destino $\rightarrow$ peso), com conjuntos de predecessores auxiliares, favorecendo grafos esparsos.

Ambas recebem no construtor o número de vértices do grafo e respeitam o mesmo contrato comportamental. A API inclui, entre outros métodos: contagem de vértices e arestas; `addEdge`, `removeEdge` e `hasEdge`; consultas de sucessor e predecessor; predicados estruturais (`isDivergent`, `isConvergent`, `isIncident`); cálculo de grau de entrada e de saída; definição e consulta de pesos de vértices e de arestas.

2. Restrições e Tratamento de Erros

A operação `addEdge` é idempotente (chamadas repetidas com o mesmo par de vértices não criam arestas duplicadas) e proíbe laços. São lançadas exceções específicas para índices de vértices inválidos e para operações inconsistentes.

3. Demonstração

Acompanha a biblioteca um programa de demonstração, executável em terminal, que percorre as principais funcionalidades da API construindo o mesmo grafo de exemplo nas duas representações e comparando os resultados lado a lado: criação dos grafos, inserção e remoção de arestas, consulta de sucessores e predecessores, cálculo de graus, definição e consulta de pesos de vértices e de arestas, verificações de grafo vazio, completo e conectado, e exportação para o Gephi.

4. Divisão do Trabalho

A Tabela 1 apresenta a divisão de responsabilidades entre os integrantes do grupo nesta etapa.

Tabela 1. Divisão de responsabilidades do trabalho.

Nome	Responsabilidades do trabalho
Luan Inaibes	Implementação da classe abstrata <code>AbstractGraph</code> e da representação por matriz de adjacência.
Otávio Santos	Implementação da representação por lista de adjacência e da hierarquia de exceções.
Arthur Luis	Programa de demonstração, testes e redação do relatório.